Mahidol University International College

EGCI211 T2(2025-2026)

Professor: Mingmanas Sivaraksa

ShareSpace

Data Structures Project Report

Date: March 26, 2026

Group Members:

1. Sarina Khuratsena 6780644
2. Pennueng Kasemsuppakit 6880207
3. Yotsarun Kiatmaneesri 6880822

1. Introduction & Background

ShareSpace is a community-oriented item donation and claiming system built in C++. The program serves as a digital platform that connects donors with recipients, allowing people to donate items they no longer need and for others to claim them freely. The theme of the project is community sharing — inspired by the idea that one person's unused item can be another person's treasure.

The system was designed to simulate a real-world free-exchange or charity shelf scenario. Users can donate items in various categories such as Electronics, Books, Clothing, Furniture, and General items. Each item is recorded with its name, condition, donor name, and category. The program enables browsing, searching, claiming, and tracking of all donated items in an intuitive menu-driven console interface.

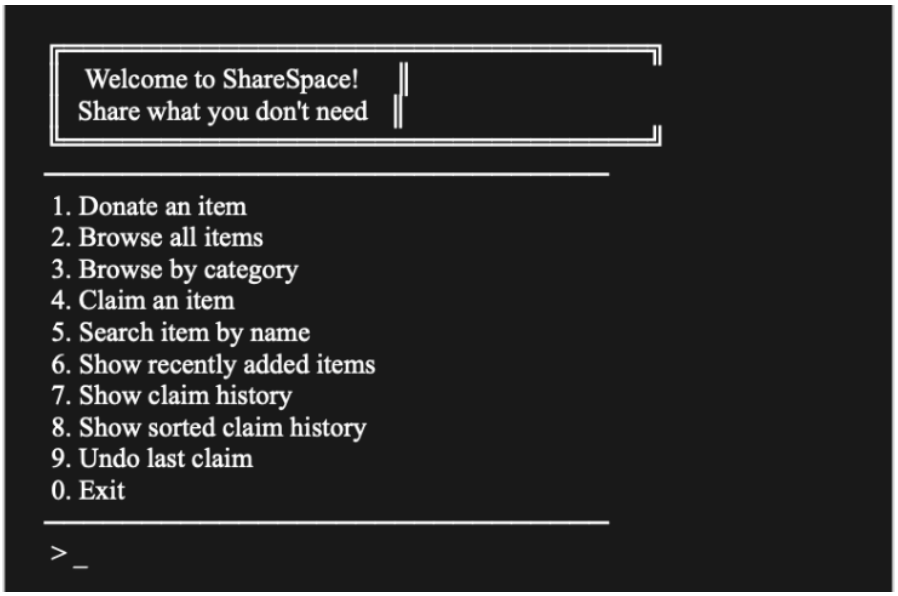
2. Requirements

The program fulfills the following functional requirements:

- Donation Management: Users can donate items by providing the item name, their name, condition, and category. Category-specific attributes (e.g., brand for Electronics, author for Books, size for Clothing) are also captured.
- Browsing: All available items can be browsed in alphabetical order (A to Z). Items can also be filtered by category.
- Claiming: Users can claim an unclaimed item by entering their name. The system uses pagination (5 items per page) for easier navigation.
- Search: Items can be searched using partial, case-insensitive keyword matching.
- Recent Items: The 10 most recently donated items are tracked and displayed in a dedicated view.
- Claim History: A full log of all claim interactions is maintained and can be displayed.
- Sorted Claim History: Claim history is maintained in a separate sorted linked list ordered alphabetically by item name.
- Undo: The most recent claim can be undone, restoring the item to available status.

3. Program Overview

To align with the project theme of community sharing and sustainability, the team decided to build a program called ShareSpace — a platform where users can donate items they no longer need and allow others to claim them for free. The program tracks all donated items, supports searching and browsing by category, maintains a claim history, and allows the most recent claim to be undone.



Program Flow

1. The program greets the user and displays the main menu with 9 options.

```
int main() {
    cout << "╔═══════════════════════════════════╗" << endl;
    cout << "‖   Welcome to ShareSpace!     ‖" << endl;
    int choice;
    do {
        // display menu options 1–9
        cin >> choice;
    } while (choice != 0);
}
```

2. Option 1 : Donate an item: the user enters the item name, their name, condition (New / Good / Fair), and category. Depending on the category selected, the program collects extra

details and brand for Electronics, author for Books, and size for Clothing. The item is inserted into the BST keyed by name, and also added to the recent items linked list.

```
void addItem() {
   string name, donor;
   string cond = pickCondition();  // New / Good / Fair
   // category selection → create correct subclass object
   if (cat == 1) item = new Electronics(name, cond, donor, brand);
   else if (cat == 2) item = new Book(name, cond, donor, author);
   else if (cat == 3) item = new Clothing(name, cond, donor, size);
   root = insertBST(root, item, inserted);
   addToRecent(item);
}
```

3. Options 2 & 3 : Browse all items / Browse by category: the BST is traversed inorder to retrieve all items in alphabetical order (A to Z). Option 3 additionally filters results by the selected category.

4. Option 4 : Claim an item: unclaimed items are listed with pagination (5 per page). Once selected, the item is marked as claimed, the claim is logged in the claim history vector and the sorted linked list, and the claim is pushed onto the undo stack.

```
target->isClaimed = true;
claimLogs.push_back(ClaimLog(target->name, target->donorName, claimer));
sortedClaimHistory.insert(target->name, target->donorName,
                          target->getCategory(), time(nullptr));
undoStack.push(target->name, claimer);
```

5. Option 5 : Search by name: performs a partial, case-insensitive keyword search across all items in the BST. Matching items are listed, and the user can claim directly from the results.

6. Option 6 : Show recently added items: displays the 10 most recently donated items stored in the linked list, with an option to claim from this view.

7. & 8. Options 7 & 8 : Claim history: Option 7 displays the raw claim log in insertion order. Option 8 displays the sorted claim history via the sorted linked list, which maintains alphabetical order by item name at all times.

9. Option 9: Undo last claim: pops the top entry from the undo stack, finds the item in the BST, and restores its status to unclaimed.

```
void undoLastClaim() {
    string itemName, claimerName;
    if (undoStack.pop(itemName, claimerName)) {
        Item* item = searchBST(root, itemName);
        item->isClaimed = false;
        claimLogs.pop_back();
    }
}
```

4. Data Structures Used

The program employs four distinct data structures, each chosen to best support a specific functionality of the system.

4.1 Binary Search Tree (BST)

The Binary Search Tree is the primary storage structure for all donated items. Each BST node holds a pointer to an Item object, with left and right child pointers. Items are keyed by their name in alphabetical order. This allows efficient insertion and lookup operations.

The BST supports inorder traversal, which yields all items in alphabetical order (A to Z) — this is used in the Browse All, Browse by Category, and Search functions. The BST also enables O(log n) average-case search by exact item name, which is utilized when claiming or undoing a claim.

4.2 Linked List (Recent Items)

A singly linked list is used to track the 10 most recently donated items. New items are always added at the front (head) of the list. When the list exceeds 10 items, the oldest item (at the tail) is automatically removed using the removeOldest operation, which traverses to the second-to-last node.

Each node in this list stores the item name, donor name, category, and a Unix timestamp. This structure provides O(1) insertion at the front and supports constant-time display of recent activity.

4.3 Sorted Linked List (Claim History)

A sorted linked list maintains the history of all claimed items, sorted alphabetically by item name. Every time an item is claimed, it is inserted into the correct alphabetical position in this list. The insert operation traverses the list to find the proper position, resulting in O(n) insertion time.

This structure ensures that the sorted claim history can always be displayed in A to Z order without any additional sorting step, making display efficient at O(n).

4.4 Stack (Undo Operations)

A stack is used to support the undo last claim feature. Each time an item is successfully claimed, the item name and claimer name are pushed onto the stack. When the user requests an undo, the

stack pops the most recently pushed entry, restores the item's status to unclaimed in the BST, and removes the last entry from the claim log vector.

The stack follows a Last-In, First-Out (LIFO) principle, which naturally matches the semantics of an undo operation — only the most recent action can be reversed. The internal implementation uses a singly linked list of StackNodes.

5. Object-Oriented Design

The program applies key OOP principles including inheritance, polymorphism, and encapsulation through a class hierarchy rooted at the Item base class.

5.1 Class Hierarchy

Class	Inherits From	Extra Attribute	Category
Item	(Base Class)	name, condition, donorName, isClaimed	General
Electronics	Item	brand	Electronics
Book	Item	author	Book
Clothing	Item	size	Clothing
Furniture	Item	(none)	Furniture

Each subclass overrides the displayInfo() virtual method to include its category-specific field, and overrides getCategory() to return its appropriate category string. This allows all item types to be stored as Item* pointers in the BST and handled polymorphically throughout the system.

6. System Features & Functionality

The program provides a console-based menu with 9 operational options, each backed by the data structures described above.

Menu Option	Feature	Data Structure Used

1	Donate an item	BST (insert) + Linked List (add to recent)
2	Browse all items	BST (inorder traversal)
3	Browse by category	BST (inorder traversal, filtered)
4	Claim an item	BST (inorder) + Stack (push) + Sorted Linked List (insert)
5	Search item by name	BST (inorder + partial string match)
6	Show recently added	Linked List (display)
7	Show claim history	vector<ClaimLog> (display)
8	Show sorted claim history	Sorted Linked List (display)
9	Undo last claim	Stack (pop) + BST (search)

7. Analysis

The choice of data structures was deliberate and tailored to the operational requirements of each feature. The table below summarizes the time complexity of the core operations.

Operation	Data Structure	Time Complexity
Insert new item	BST	O(log n) average, O(n) worst
Search item by name	BST	O(log n) average
Browse all items (A-Z)	BST inorder	O(n)
Add to recent items	Linked List	O(1) front insertion
Trim oldest recent item	Linked List	O(n) tail traversal
Insert claim record (sorted)	Sorted Linked List	O(n) traversal
Display sorted history	Sorted Linked List	O(n)
Push claim to undo stack	Stack	O(1)
Pop (undo last claim)	Stack	O(1)

The BST is central to the system and handles the majority of item operations. Using a BST over a simple array or vector allows the system to maintain sorted order naturally via inorder traversal, avoiding the need for explicit sorting. However, in worst case scenarios (e.g., items donated in alphabetical order), the BST could degenerate to O(n) per operation due to an

unbalanced tree. A self-balancing BST such as an AVL tree could address this in a production system.

The stack's O(1) push and pop operations make the undo feature extremely efficient, regardless of how many items are in the system. The sorted linked list trades insertion speed (O(n)) for always-sorted retrieval (O(n) display with no additional sorting), which is acceptable given the read-heavy nature of the claim history view.

8. Summary

ShareSpace demonstrates the practical application of four fundamental data structures within a unified, real-world inspired C++ project. The Binary Search Tree provides efficient alphabetically ordered item storage. The singly linked list tracks recent donations with constant-time additions. The sorted linked list maintains claim history in permanent alphabetical order. The stack enables undo functionality with minimal overhead.

The OOP design, using a base Item class and four subclasses, allows the system to handle multiple item types uniformly while still capturing category-specific data. Polymorphism through virtual methods ensures that displaying or categorizing any item requires no special-case logic in the higher-level functions.

Through this project, the team gained hands-on experience in selecting and implementing the most appropriate data structure for each specific use case, understanding the trade-offs between time complexity and code simplicity, and applying OOP principles to build a maintainable and extensible C++ program.

9. Sources

C++ STL Reference
https://cppreference.com/

Binary Search Tree — GeeksForGeeks
https://www.geeksforgeeks.org/binary-search-tree-data-structure/

Stack Data Structure — GeeksForGeeks
https://www.geeksforgeeks.org/stack-data-structure/